

МАШИННОЕ ОБУЧЕНИЕ

для начинающих

Учебник для тех, кто хочет научить компьютер думать

Практический курс · от первых данных до своего проекта
Python · Pandas · Scikit-learn · нейросети · LLM

Содержание

Как читать эту книгу

Эта книга — не сборник формул и не толстый справочник, который страшно открывать. Это **маршрут**: от «я вообще не понимаю, что такое машинное обучение» до момента, когда ты запускаешь собственное приложение и отправляешь ссылку на него друзьям.

Ты будешь не читать про машинное обучение, а заниматься им. Почти в каждой главе есть код, который можно запустить прямо в браузере, и задание, которое стоит сделать самому.

Правило простое: если можно попробовать — попробуй.

Что нужно знать заранее

Мы предполагаем, что ты уже писал простые программы на Python: знаешь, что такое переменная, список, цикл и функция. Если ты когда-нибудь делал запрос к базе данных (SQL) — вообще отлично, это пригодится. Всё остальное объясняется с нуля.

Как устроена книга

Книга разбита на шесть частей — мы называем их фазами. Каждая следующая фаза опирается на предыдущую, поэтому читай по порядку.

Шесть фаз пути

Фаза 1 — Данные и ML-мышление. Учимся смотреть на данные и находить в них закономерности.

Фаза 2 — Классический ML. Строим первые модели, которые предсказывают.

Фаза 3 — Нейросети. Разбираемся, как машина «видит» и учится на примерах.

Фаза 4 — Большие языковые модели (LLM). Понимаем, как работает то, что стоит за ChatGPT.

Фаза 5 — AutoML и веб-приложения. Превращаем модель в продукт с кнопкой.

Фаза 6 — Финальный проект. Делаем и защищаем что-то своё.

Значки, которые встретятся в тексте

Аналогия

Так помечены сравнения из обычной жизни — они помогают понять сложную идею быстро.

Внимание, частая ошибка

Так отмечены места, где почти все спотыкаются. Прочитай внимательно — сэкономишь часы.

Полезно знать

Дополнение или уточнение, которое углубляет понимание, но не обязательно для первого прохода.

Где запускать код

Весь код в книге рассчитан на **Google Colab** — бесплатную среду, которая работает прямо в браузере. Ничего устанавливать не нужно: заходишь на colab.research.google.com, создаёшь новый блокнот и вставляешь код. Всё считается на серверах Google.

Готов? Тогда начнём с главного вопроса: чем машинное обучение вообще отличается от обычного программирования.

ФАЗА 1

Данные и ML-мышление

Учимся видеть закономерности в данных — фундамент всего, что будет дальше.

Глава 1. «Магия» машинного обучения

Две разные идеи: правила и обучение

Представь, что тебя попросили написать программу, которая отличает кошку от собаки на фотографии. Как бы ты это сделал обычным способом — через правила?

Ты бы начал: «Если уши треугольные и торчат вверх — это кошка. Если морда вытянутая — собака...» И очень быстро увяз бы. А вислоухая кошка? А чихуахуа с торчащими ушами? А фото сбоку? Правил становятся тысячи, и они всё равно не работают.

Машинное обучение переворачивает задачу. Вместо того чтобы писать правила, ты показываешь машине **тысячи примеров** — «вот кошка», «вот собака», «вот снова кошка» — и она сама находит закономерности, которые отличают одно от другого. Правила она выводит без тебя.

Аналогия: рецепт и дегустация

Обычное программирование — это рецепт: ты записываешь точные шаги, и повар их выполняет. Машинное обучение — это дегустатор, который попробовал сотни блюд и теперь на вкус угадывает, солёное оно или сладкое, хотя рецепта ему никто не давал.

Определение своими словами

Главная мысль главы

Машинное обучение — это способ научить компьютер решать задачу не через записанные правила, а через примеры, из которых он сам выводит закономерности (паттерны).

Что такое «паттерн в данных»

Паттерн — это устойчивая закономерность. Люди находят паттерны постоянно, даже не замечая. Ты знаешь, что после серых туч часто идёт дождь. Никто не давал тебе формулу «туча → дождь» — ты вывел её из наблюдений. Именно этим занимается модель машинного обучения, только с числами.

Например, в данных о жилье может прятаться паттерн: «чем больше площадь, тем выше цена». В данных о фильмах: «фильмы с рейтингом выше 8 чаще собирают большую кассу». Модель ищет такие связи и потом использует их, чтобы делать **предсказания** на новых данных.

Типы задач машинного обучения

Почти всё, что мы будем делать, сводится к трём большим типам задач. Разберём каждый на понятном примере.

1. Классификация — «к какой категории?»

Модель выбирает одну из нескольких категорий. Спам или не спам. Кошка, собака или птица. Положительный отзыв или отрицательный. Ответ — это метка (label).

2. Регрессия — «сколько?»

Модель предсказывает число. Какой будет цена квартиры. Сколько баллов наберёт ученик. Какая температура будет завтра. Ответ — это число на шкале, а не категория.

3. Кластеризация — «кто на кого похож?»

Модель сама, без подсказок, разбивает данные на группы похожих объектов. Например, находит среди покупателей группы со схожим поведением. Здесь никто заранее не говорит модели правильный ответ — она группирует сама.

Тип задачи	Вопрос	Пример ответа
Классификация	К какой категории?	спам / не спам
Регрессия	Сколько? Какое число?	цена = 4 200 000
Кластеризация	Кто на кого похож?	группа А, В, С

Внимание, частая ошибка

Классификацию и регрессию путают чаще всего. Запомни разницу по вопросу: если ответ — это категория (одна из нескольких), это классификация. Если ответ — число на непрерывной шкале, это регрессия. «Сдаст экзамен / не сдаст» — классификация. «Сколько баллов наберёт» — регрессия.

Обучение с учителем и без учителя

Классификация и регрессия — это обучение с учителем (supervised): в данных есть правильные ответы, на которых модель тренируется. Кластеризация — обучение без учителя (unsupervised): правильных ответов нет, модель ищет структуру сама. В этом курсе мы почти всё время работаем с обучением с учителем — оно нагляднее и понятнее в начале пути.

Попробуй сам: игра «Угадай правило»

Эту игру можно сыграть без единой строчки кода — с другом или всем классом. Она показывает суть машинного обучения буквально за пять минут.

1. Один игрок загадывает секретное правило, например: «число проходит, если оно чётное».
2. Второй игрок называет числа: 4 → проходит, 7 → не проходит, 10 → проходит...
3. С каждым ответом второй игрок уточняет свою гипотезу о правиле — ровно как модель уточняет закономерность на новых примерах.
4. Когда второй уверен — он называет правило. Угадал? Значит «обучение» удалось.

Ты только что сработал как модель машинного обучения: получал примеры с ответами и постепенно выводил скрытое правило. Именно это делает алгоритм — только с тысячами примеров и очень быстро.

Проверь себя

1. Чем машинное обучение отличается от программирования правилами? Объясни в одном предложении.
2. Приведи по одному своему примеру задачи классификации и задачи регрессии.
3. Задача «разбить учеников школы на группы по интересам, не зная заранее какие» — это какой тип обучения?

Глава 2. Данные вокруг нас: знакомство с Pandas

Машинное обучение начинается не с моделей, а с данных. Прежде чем что-то предсказывать, нужно научиться данные открывать, читать и понимать. Главный инструмент для этого в Python называется Pandas.

Аналогия: Pandas — это Excel внутри Python 💡

Если ты когда-нибудь работал с таблицей в Excel или Google Таблицах — ты уже почти понимаешь Pandas. Это тот же самый лист с колонками и строками, только вместо мышки ты управляешь им кодом. Кода бояться не нужно: команд немного, и они читаются почти как английские фразы.

Три главных слова: датасет, признак, целевая переменная

Датасет — это таблица с данными. Каждая **строка** — один объект (например, один пассажир корабля). Каждый **столбец** — одна характеристика объекта, её называют **признаком** (feature): возраст, пол, цена билета.

Среди столбцов часто есть один особенный — тот, который мы хотим предсказывать. Его называют **целевой переменной** (target). Например, «выжил пассажир или нет». Все остальные признаки — это подсказки, по которым модель будет угадывать целевую переменную.

	Возраст (признак)	Пол (признак)	Выжил (цель)
Пассажир 1	22	муж.	нет
Пассажир 2	38	жен.	да
Пассажир 3	26	жен.	да

Первый код: открываем датасет

Загрузим известный учебный датасет о пассажирах «Титаника» — мы будем возвращаться к нему на протяжении нескольких глав. Вставь этот код в новую ячейку Google Colab и запусти.

```
# Импортируем библиотеку Pandas под коротким именем pd
import pandas as pd

# Загружаем датасет "Титаник" прямо из интернета по ссылке
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url) # df — сокращение от DataFrame ("таблица")

# Показываем первые 5 строк таблицы
df.head()
```

Ожидаемый результат: таблица из 5 строк с колонками Name, Sex, Age, Survived и другими.

Три команды, чтобы понять любой датасет

Как только данные загружены, три команды дают полную картину. Запускай каждую в отдельной ячейке.

df.head() — как выглядят данные

Показывает первые несколько строк. Это первое, что делают всегда: буквально посмотреть глазами, что за таблица перед тобой, какие есть колонки и что в них лежит.

df.info() — что за колонки и нет ли пропусков

```
df.info()
```

Показывает список колонок, их тип (число или текст) и сколько значений заполнено. Если в колонке Age заполнено 714 значений из 891 — значит, 177 пропущено. Это важный сигнал, к которому мы вернёмся в главе 4.

df.describe() — статистика по числам

```
df.describe()
```

Для каждой числовой колонки показывает среднее, минимум, максимум и другие величины. Один взгляд — и ты знаешь, что средний возраст пассажира около 30 лет, а самому младшему меньше года.

Фильтрация и группировка: привет из SQL

Если ты знаком с SQL, следующие операции покажутся родными. Pandas умеет то же самое, что запросы к базе данных, только внутри Python.

Фильтрация — как WHERE

```
# Оставить только пассажиров женского пола
women = df[df["Sex"] == "female"]
women.head()
```

Это прямой аналог SQL-запроса `SELECT * FROM df WHERE Sex = 'female'`. Внутри квадратных скобок мы пишем условие, и остаются только строки, для которых оно истинно.

Группировка — как GROUP BY

```
# Средняя доля выживших отдельно для мужчин и женщин
df.groupby("Sex")["Survived"].mean()
```

Эта одна строка отвечает на настоящий вопрос: кто выживал чаще? Результат обычно поражает — среди женщин доля выживших намного выше. Это и есть паттерн в данных, найденный за секунду.

Внимание, частая ошибка ⚠

Легко перепутать одиночное = (присваивание) и двойное == (сравнение). В условии фильтра всегда двойное: `df["Sex"] == "female"`. Одиночное = здесь вызовет ошибку.

Домашнее задание

Зайди на сайт Kaggle (kaggle.com/datasets) и найди датасет на тему, которая тебе интересна: игры, музыка, спорт, кино. Загрузи его в Colab через `pd.read_csv` и ответь на три вопроса:

- Сколько в нём строк и столбцов? (подсказка: `df.shape`)
- Какие есть признаки и есть ли среди них пропуски? (`df.info()`)
- Найди один интересный факт с помощью `groupby` — например, среднее значение чего-то по категориям.

Глава 3. Визуализация и разведка данных (EDA)

Таблица из тысяч чисел ничего не говорит человеку напрямую. А один хороший график — говорит сразу всё. Процесс, когда мы разглядываем данные через графики и ищем в них закономерности, называется **разведочным анализом данных** (EDA, Exploratory Data Analysis). Это как осмотреться на новом месте, прежде чем действовать.

Три графика, которые нужны почти всегда

Гистограмма — как распределены значения

Гистограмма показывает, каких значений в данных много, а каких мало. Например, каких возрастов среди пассажиров больше всего.

```
import matplotlib.pyplot as plt # библиотека для графиков

# Гистограмма возрастов пассажиров
df["Age"].hist(bins=20)         # bins — на сколько столбиков делить
plt.xlabel("Возраст")
plt.ylabel("Сколько пассажиров")
plt.title("Распределение возрастов")
plt.show()
```

Ожидаемый результат: столбчатая диаграмма — видно, что больше всего пассажиров в возрасте 20–35 лет.

Диаграмма рассеяния (scatter plot) — связь двух признаков

Каждая точка — один объект. По одной оси один признак, по другой — второй. Так видно, связаны ли они. Если точки складываются в линию — связь есть.

```
# Есть ли связь между возрастом и ценой билета?
df.plot.scatter(x="Age", y="Fare")
plt.show()
```

Тепловая карта корреляций (heatmap) — что с чем связано

Показывает сразу для всех пар признаков, насколько сильно они связаны. Понадобится Seaborn — библиотека красивых графиков поверх Matplotlib.

```
import seaborn as sns

# Считаем корреляции только по числовым колонкам
corr = df.corr(numeric_only=True)

# Рисуем тепловую карту: чем ярче клетка, тем сильнее связь
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.show()
```

Что такое корреляция

Аналогия: качели

Корреляция — это число от -1 до +1, показывающее, как связаны два признака. +1: когда один растёт, другой тоже растёт (как рост и вес). -1: когда один растёт, другой падает (как цена билета и класс каюты, где 1-й класс дороже 3-го). Около 0: связи нет, ведут себя независимо.

Внимание, частая ошибка ⚠

Корреляция — это не причина! Если мороженое и солнечные ожоги растут вместе, это не значит, что мороженое вызывает ожоги. Просто оба зависят от жаркой погоды. Всегда спрашивай себя: нет ли третьей причины?

Главное правило EDA: график без вывода бесполезен

Построить график — половина дела. Вторая половина — сформулировать словами, что он показывает. Возьми за привычку: под каждым графиком писать вывод одним предложением. «На гистограмме видно, что большинство пассажиров — молодые люди 20–35 лет».

Мини-конкурс: найди самый необычный инсайт

Возьми свой датасет из прошлого задания и построй на нём три разных графика. Для каждого напиши вывод словами. Затем выбери самый неожиданный факт — то, чего ты не ожидал увидеть. Это и есть инсайт: то, что данные рассказали тебе, а ты не знал заранее.

Проверь себя ⓘ

1. Какой график выберешь, чтобы понять, как распределён возраст в классе?
2. Корреляция между двумя признаками равна $-0,8$. Что это значит?
3. Почему нельзя говорить «признак А вызывает признак В» только на основании высокой корреляции?

Глава 4. Мини-проект «Расскажи о данных»

Пришло время собрать всё из первой фазы вместе. Это твой первый мини-проект — небольшое, но полное исследование, которое можно показать друзьям и родителям. Здесь мы также научимся справляться с двумя частыми проблемами реальных данных: пропусками и выбросами.

Проблема 1: пропущенные значения

Реальные данные почти всегда неполные. В «Титанике» у многих пассажиров не указан возраст. Модель не умеет работать с пустыми клетками, поэтому пропуски нужно как-то обработать.

```
# Сколько пропусков в каждой колонке?
df.isnull().sum()

# Вариант 1: удалить строки с пропусками (осторожно — теряем данные)
df_clean = df.dropna()

# Вариант 2: заполнить пропуски средним значением (чаще лучше)
df["Age"] = df["Age"].fillna(df["Age"].median())
```

Полезно знать

Заполнять пропуски медианой (серединным значением), а не средним, безопаснее: медиана не «съезжает» из-за нескольких очень больших или очень маленьких значений.

Проблема 2: выбросы

Выброс (outlier) — это значение, которое сильно выбивается из общего ряда. Например, если у одного пассажира указана цена билета в 10 раз выше всех остальных. Выбросы могут быть как ошибкой ввода, так и настоящим редким случаем. Найти их помогает график типа «ящик с усами» (boxplot).

```
import matplotlib.pyplot as plt

# Ящик с усами по цене билета: точки далеко от "ящика" — выбросы
df.boxplot(column="Fare")
plt.show()
```

Структура твоего мини-проекта

Оформи исследование как небольшую презентацию из трёх частей. На выступление хватит трёх минут.

1. О данных. Что за датасет, сколько строк и столбцов, какие признаки. Один слайд.
2. Что нашёл. 2–3 графика с выводами словами и главный инсайт. Один-два слайда.
3. Вывод. Один содержательный факт, который данные рассказали тебе. Один слайд.

Как оценивается мини-проект

Мини-проект оценивается по простой рубрике — она помогает понять, что важно, а не для отметки.

Критерий	Баллы	Что оценивается
Постановка	0–2	Понятно, о каких данных речь
Работа с данными	0–3	Обработаны пропуски и выбросы
Визуализация	0–2	Графики уместны и подписаны
Объяснение	0–3	Есть вывод словами и инсайт

Главная мысль фазы 1

Ты научился главному навыку любого специалиста по данным: взять сырую таблицу, привести её в порядок, разглядеть в ней закономерности и рассказать о них человеческим языком. Теперь мы готовы сделать следующий шаг — научить машину предсказывать.

ФАЗА 2

Классический ML

Строим первые модели, которые предсказывают, и учимся честно проверять их качество.

Глава 5. KNN: алгоритм ближайших соседей

В первой фазе мы разглядывали данные. Теперь научим машину предсказывать. Начнём с самого понятного алгоритма во всём машинном обучении — метода k ближайших соседей, коротко KNN (k-Nearest Neighbors).

Аналогия: скажи мне, кто твой сосед

KNN — это «ленивый ученик». Он ничему не учится заранее. Когда нужно классифицировать новый объект, он просто смотрит: а на кого этот объект похож среди тех, кого я уже знаю? Находит несколько самых близких «соседей» и повторяет их ответ. Скажи мне, кто твои соседи, — и я скажу, кто ты.

Как это работает по шагам

Представь точки на плоскости: синие — «сдал экзамен», красные — «не сдал». Появляется новая точка без цвета. Что делает KNN?

1. Измеряет расстояние от новой точки до всех известных точек.
2. Выбирает k ближайших (например, $k = 5$ — пять самых близких).
3. Смотрит, какого цвета большинство из этих пяти соседей.
4. Присваивает новой точке цвет большинства. Голосование окончено.

Параметр K — сколько соседей спрашивать

K — это главная настройка алгоритма. Она задаёт, скольких соседей мы опрашиваем. И выбор K сильно меняет поведение модели.

Значение K	Что происходит	Риск
K = 1	Смотрим только на одного ближайшего соседа	Модель шумит, реагирует на случайные точки
K средний (5–15)	Голосует небольшая группа соседей	Обычно оптимально
K очень большой	Голосует почти весь датасет	Модель «размывается», теряет детали

Полезно знать

K обычно берут нечётным (3, 5, 7...) — чтобы при голосовании не случилось ничьей. Как правильно подбирать K по-честному, разберём в следующей главе.

Живая игра без кода

Расставьте по классу учеников двух «типов» — например, любителей спорта и любителей музыки — в разных углах комнаты. Пусть новый ученик встанет где-то посередине и посмотрит на трёх ближайших к себе. Кого больше рядом — к той группе KNN его и отнесёт. Вы только что стали живым алгоритмом.

Первый классификатор в коде

Соберём KNN на данных о пассажирах «Титаника»: предскажем, выжил пассажир или нет, по его возрасту и цене билета.

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
```

```
# Загружаем данные и убираем строки с пропусками (пока просто так)
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url).dropna(subset=["Age", "Fare", "Survived"])

# X — признаки (по чему предсказываем), y — цель (что предсказываем)
X = df[["Age", "Fare"]]
y = df["Survived"]

# Создаём модель: спрашиваем 5 ближайших соседей
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X, y)          # "обучаем" — KNN просто запоминает точки

# Предсказываем для нового пассажира: возраст 25, билет за 30
print(model.predict([[25, 30]]))  # 1 = выжил, 0 = нет
```

Ожидаемый результат: [0] или [1] — прогноз для выдуманного пассажира.

Внимание, частая ошибка ⚠

KNN измеряет расстояние — значит, ему важен масштаб признаков. Возраст (0–80) и цена билета (0–500) в разных диапазонах, и цена «перевесит» возраст просто потому, что числа крупнее. Это лечится масштабированием признаков — к нему мы вернёмся в главе 10, когда соберём Pipeline.

Проверь себя ⓘ

1. Почему KNN называют «ленивым» алгоритмом?
2. Что произойдёт с моделью, если задать $K = 1$?
3. Почему KNN чувствителен к масштабу признаков?

Глава 6. Честная проверка: train/test, baseline и утечка данных

Это, возможно, самая важная глава всего курса. Здесь речь не о новом алгоритме, а о том, как понять — хороша модель или мы себя обманываем. Без этого любой красивый результат ничего не стоит.

Разделение на train и test

Аналогия: зазубрил или понял 💡

Представь ученика, который выучил ответы к конкретным задачам наизусть. На этих задачах он получит 100%. Но дай ему новую задачу — и он провалится. Он зазубрил, а не понял. Модель ведёт себя точно так же: если проверять её на тех же данных, на которых она училась, она покажет отличный результат, но это ничего не говорит о новых данных.

Поэтому данные всегда делят на две части. **Обучающая выборка (train)** — на ней модель учится. **Тестовая выборка (test)** — её модель никогда не видела при обучении, и именно на ней мы честно проверяем качество. Обычно берут 80% на обучение и 20% на тест.

```
from sklearn.model_selection import train_test_split

# test_size=0.2 → 20% данных уходит в тест
# random_state=42 → фиксируем случайность, чтобы результат повторялся
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)          # учимся ТОЛЬКО на train

from sklearn.metrics import accuracy_score
pred = model.predict(X_test)         # проверяем на невиданном test
print("Accuracy:", accuracy_score(y_test, pred))
```

Ожидаемый результат: доля правильных ответов на тесте, например 0.68.

Внимание, частая ошибка ⚠️

Проверять модель на обучающих данных — ошибка номер один у новичков. Результат будет завышенным и бесполезным. Правило простое: тест модель не видит до самого конца. И всегда ставь `random_state` — иначе при каждом запуске разбиение будет разным, и результаты не сравнить.

Baseline: с чем вообще сравнивать?

Допустим, модель дала accuracy 0,68. Это хорошо или плохо? Ответить невозможно, пока нет **точки отсчёта**. Такой точкой служит baseline — самая тупая модель, какую можно придумать. Например: «всегда предсказывай самый частый класс».

Если на «Титанике» погибло 62% пассажиров, то модель, которая тупо всем ставит «погиб», получит accuracy 0,62. Значит, наши 0,68 — это плюс всего 6 пунктов к тупому угадыванию. Baseline мгновенно показывает, стоит ли модель вообще чего-то.

```
from sklearn.dummy import DummyClassifier
```

```
# "most_frequent" — всегда предсказывает самый частый класс
baseline = DummyClassifier(strategy="most_frequent")
baseline.fit(X_train, y_train)
print("Baseline accuracy:", baseline.score(X_test, y_test))
```

Data leakage — коварная утечка данных

Аналогия: подсмотреть ответы 💡

Утечка данных (data leakage) — это когда модель случайно «подсматривает» в тестовые данные или в информацию, которой в реальности не будет. Это как если бы ученик готовился к экзамену по украденным заранее ответам: на экзамене он блеснёт, а знаний нет.

Классический пример утечки: предсказываем, вернёт ли клиент кредит, и случайно оставляем в признаках колонку «сумма штрафа за невозврат». Но штраф появляется только после невозврата! В реальный момент предсказания его ещё нет. Модель покажет фантастическую точность на данных, но в жизни окажется бесполезной.

Полезно знать ⓘ

Ещё один частый источник утечки — масштабировать или заполнять пропуски на всех данных сразу, до разделения на train/test. Тогда информация из теста «протекает» в обучение. Именно поэтому в главе 10 мы соберём Pipeline: он делает все преобразования правильно и защищает от этой ошибки автоматически.

Соревнование: честный подбор K

Теперь мы умеем проверять честно. Попробуй разные значения K и найди лучшее — но оценивай только по тесту, а не по обучению.

```
for k in [1, 3, 5, 7, 15, 30]:
    m = KNeighborsClassifier(n_neighbors=k)
    m.fit(X_train, y_train)
    acc = m.score(X_test, y_test)
    print(f"K={k}: accuracy={acc:.3f}")
```

Обрати внимание: K=1 обычно даёт идеальный результат на train, но слабый на test — это первый звоночек переобучения, о котором подробно в следующей главе.

Проверь себя ⓘ

1. Зачем делить данные на train и test?
2. Модель даёт accuracy 0,55, а baseline — 0,60. Что это говорит о модели?
3. Приведи свой пример утечки данных.

Глава 7. Деревья решений: компьютер-детектив

KNN сравнивал объекты по схожести. Дерево решений действует иначе — оно задаёт данным вопросы, шаг за шагом сужая круг, как детектив на допросе.

Аналогия: игра «20 вопросов» 💡

Ты загадал персонажа, а друг угадывает через вопросы «да/нет»: «Это человек? Он реальный? Он спортсмен?» Каждый вопрос отсекает половину вариантов. Дерево решений делает ровно это: задаёт признакам вопросы вроде «Пол = женский?», «Возраст < 12?» — и с каждым ответом приближается к предсказанию.

Узлы, ветви и листья

Дерево состоит из **узлов** (в каждом — вопрос по одному признаку), **ветвей** (ответы «да»/«нет») и **листьев** (финальные предсказания). Алгоритм сам выбирает, какой вопрос задать первым: тот, что лучше всего разделяет данные на группы. Это и есть «информативность вопроса».

Дерево в коде и его картинка

Главное достоинство деревьев — их можно нарисовать и понять глазами. Построим дерево на «Титанике».

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# max_depth=3 — ограничиваем глубину, чтобы дерево было читаемым
tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X_train, y_train)

# Рисуем дерево
plt.figure(figsize=(14, 6))
plot_tree(tree, feature_names=["Age", "Fare"],
          class_names=["погиб", "выжил"], filled=True)
plt.show()
```

Ожидаемый результат: схема дерева с вопросами в узлах и предсказаниями в листьях.

Переобучение и параметр max_depth

Аналогия: зубрёжка каждой мелочи 💡

Если дереву не ограничивать глубину, оно будет задавать вопросы, пока не запомнит каждый обучающий пример по отдельности — вплоть до «этому конкретному пассажиру ставим выжил». На train будет 100%, а на новых данных — провал. Это переобучение (overfitting): модель выучила частности вместо закономерностей.

Противоположная беда — **недообучение (underfitting)**: дерево слишком простое (например, глубина 1) и не улавливает даже явных закономерностей. Задача — найти баланс. Главный рычаг здесь — max_depth, максимальная глубина дерева.

```
# Сравним дерево без ограничения и с max_depth=3
for depth in [3, None]: # None = без ограничения
    t = DecisionTreeClassifier(max_depth=depth, random_state=42)
    t.fit(X_train, y_train)
    print(f"depth={depth}: "
```

```
f"train={t.score(X_train, y_train):.2f}, "  
f"test={t.score(X_test, y_test):.2f}")
```

Ты увидишь: без ограничения train близок к 1,00, а test заметно ниже — классический разрыв, признак переобучения.

Минута этики

Дерево наглядно показывает, по каким признакам оно решает. А что, если модель принимает важные решения о людях — по признаку пола или возраста? Это повод для разговора: какие признаки использовать этично, а какие могут закрепить несправедливость. Модель делает то, чему её научили данные, — и ответственность на человеке.

Проверь себя

1. Что такое переобучение? Как его заметить по train и test?
2. Какой параметр ограничивает сложность дерева?
3. Чем недообучение отличается от переобучения?

Глава 8. Регрессия: предсказываем числа

До сих пор мы предсказывали категории — выжил или нет. Но огромный класс задач требует предсказать число: цену, температуру, оценку. Это регрессия.

Классификация против регрессии

Классификация отвечает на вопрос «какая категория?» (кошка/собака). Регрессия отвечает на вопрос «сколько?» (сколько будет стоить, сколько баллов). Если ответ — число на непрерывной шкале, это регрессия.

Линейная регрессия: линия через облако точек

Аналогия: линейка через точки

Нанеси на график точки: по горизонтали — часы учёбы, по вертикали — оценка за экзамен. Точки образуют «облако», уходящее вверх: больше учишься — выше оценка. Линейная регрессия проводит через это облако прямую линию так, чтобы она проходила максимально близко ко всем точкам. По этой линии можно предсказать оценку для любого числа часов.

Регрессия в коде на синтетических данных

Возьмём простые данные «часы учёбы → баллы» — они идеально показывают идею.

```
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

# Синтетические данные: часы учёбы и баллы (с небольшим шумом)
hours = np.array([1, 2, 3, 4, 5, 6, 7, 8]).reshape(-1, 1)
scores = np.array([20, 35, 40, 55, 60, 72, 80, 88])

model = LinearRegression()
model.fit(hours, scores) # находим лучшую прямую

# Рисуем точки и линию модели
plt.scatter(hours, scores, label="реальные данные")
plt.plot(hours, model.predict(hours), color="red", label="линия модели")
plt.xlabel("Часы учёбы"); plt.ylabel("Баллы"); plt.legend()
plt.show()

# Предсказание: сколько баллов при 5.5 часах учёбы?
print("Прогноз:", model.predict([[5.5]]))
```

Ожидаемый результат: график с точками и красной линией + прогноз около 66 баллов.

Метрики регрессии: MAE и MSE

В классификации мы считали долю правильных ответов. В регрессии ответ — число, и «правильно/неправильно» не подходит: важно, насколько мы промахнулись. Для этого две главные метрики.

Метрика	Что это	Особенность
MAE	Средняя абсолютная ошибка — на сколько в среднем промахиваемся	В единицах задачи (баллы, рубли). Легко понять
MSE	Средний квадрат ошибки	Сильно наказывает крупные промахи

```
from sklearn.metrics import mean_absolute_error, mean_squared_error

pred = model.predict(hours)
print("MAE:", mean_absolute_error(scores, pred))
print("MSE:", mean_squared_error(scores, pred))
```

Полезно знать

MAE = 4 означает: в среднем модель ошибается на 4 балла. Это интуитивно понятно, потому что измеряется в тех же единицах, что и цель. MSE труднее толковать напрямую, зато она сильнее реагирует на большие ошибки — полезно, когда крупные промахи особенно нежелательны.

Проверь себя

1. «Предсказать завтрашнюю температуру» — это классификация или регрессия?
2. Что означает MAE = 3 в задаче предсказания оценок?
3. Почему MSE сильнее наказывает большие ошибки, чем MAE?

Глава 9. Случайный лес: сила ансамбля

Одно дерево решений легко переобучается и капризно. А что, если взять много разных деревьев и устроить голосование? Так рождается случайный лес (Random Forest) — один из самых надёжных алгоритмов классического ML.

Аналогия: голосование класса

Спроси одного ученика — он может ошибиться. Спроси весь класс и возьми ответ большинства — коллективный ответ почти всегда точнее. Это «мудрость толпы». Случайный лес строит сотни деревьев, каждое немного по-своему, и объединяет их голоса. Ошибки отдельных деревьев взаимно гасятся.

Почему деревья должны быть разными

Если бы все деревья были одинаковыми, голосование ничего не дало бы — они ошибались бы хором. Поэтому лес специально делает деревья разнообразными: каждое учится на случайной части данных и случайном наборе признаков. Разнообразие — источник силы ансамбля.

Эксперимент-викторина

Задайте классу вопрос с числовым ответом («сколько окон в школе?»). Запишите ответы всех и посчитайте среднее. Почти наверняка среднее по классу окажется ближе к правде, чем большинство отдельных ответов. Вы наглядно показали принцип ансамбля.

Случайный лес в коде

```
from sklearn.ensemble import RandomForestClassifier

# n_estimators — сколько деревьев в лесу
forest = RandomForestClassifier(n_estimators=100, random_state=42)
forest.fit(X_train, y_train)

print("Лес, accuracy:", forest.score(X_test, y_test))
```

Важность признаков

Лес умеет сказать, какие признаки оказались важнее для предсказания, — это **feature importance**. Мощный инструмент, чтобы понять данные.

```
import pandas as pd

# Насколько каждый признак важен для леса
importances = pd.Series(forest.feature_importances_, index=X.columns)
print(importances.sort_values(ascending=False))
```

Турнир алгоритмов

Мы изучили три классификатора. Устроим честный турнир на одних и тех же train/test и сравним.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
```

```
models = {
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Дерево": DecisionTreeClassifier(max_depth=3, random_state=42),
    "Лес": RandomForestClassifier(n_estimators=100, random_state=42),
}
for name, m in models.items():
    m.fit(X_train, y_train)
    print(f"{name}: {m.score(X_test, y_test):.3f}")
```

Чаще всего лес выигрывает — но не всегда. Иногда простая модель не хуже, а понятнее. Это важный урок: сложнее не значит лучше.

Проверь себя

1. Почему ансамбль деревьев точнее одного дерева?
2. За что отвечает параметр `n_estimators`?
3. Что показывает `feature importance`?

Глава 10. Pipeline и кросс-валидация

Правило курса

Начиная с этой главы, все модели в книге собираются через Pipeline. Это профессиональный стандарт, который исключает целый класс ошибок — в первую очередь утечку данных. Дальше без Pipeline мы не работаем.

Зачем нужен Pipeline

Аналогия: конвейер на заводе

Pipeline — это конвейер: данные входят с одного конца, проходят через одни и те же станки (заполнить пропуски → закодировать текст → отмасштабировать → модель) и выходят готовым предсказанием. Собрав конвейер один раз, ты гарантируешь, что и на обучении, и на тесте, и в реальном приложении данные обрабатываются абсолютно одинаково.

Главное: Pipeline «настраивает» преобразования только на train (например, вычисляет среднее для заполнения пропусков) и применяет их к test — а не подсматривает в test заранее. Именно ручное масштабирование до разделения было источником утечки из главы 6. Pipeline закрывает эту дыру автоматически.

Масштабирование признаков

Вспомни проблему KNN: возраст 0–80 и цена 0–500 несопоставимы по масштабу. Масштабирование приводит все признаки к сравнимому диапазону, чтобы ни один не «перевешивал» просто из-за крупных чисел. Самый частый инструмент — StandardScaler.

Собираем первый Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.neighbors import KNeighborsClassifier

# Конвейер из трёх станков:
pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")), # заполнить пропуски
    ("scale", StandardScaler()),                 # отмасштабировать
    ("model", KNeighborsClassifier(n_neighbors=5)), # модель
])

pipe.fit(X_train, y_train) # весь конвейер обучается разом
print("Pipeline accuracy:", pipe.score(X_test, y_test))
```

Теперь пропуски и масштаб обрабатываются честно, без утечки.

Разные типы столбцов: ColumnTransformer

В реальных данных числа и текст соседствуют: возраст (число) и пол (текст «male»/«female»). Числа масштабируют, текст — кодируют в числа. ColumnTransformer позволяет применить разные преобразования к разным столбцам внутри одного конвейера.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
```

```

num = ["Age", "Fare"]      # числовые столбцы
cat = ["Sex"]              # текстовые столбцы

prep = ColumnTransformer([
    ("num", StandardScaler(), num),
    ("cat", OneHotEncoder(), cat),  # текст → числа (0/1)
])

full = Pipeline([("prep", prep),
                  ("model", RandomForestClassifier(random_state=42))])

```

Кросс-валидация: одна проверка — ненадёжно

Аналогия: одна контрольная или четыре 💡

Оценить ученика по одной контрольной ненадёжно — вдруг попался неудачный вариант. Четыре контрольные по разным темам дают честную картину. Кросс-валидация делает то же: делит данные на несколько частей (folds), по очереди обучает на одних и проверяет на другой, а потом усредняет результат.

```

from sklearn.model_selection import cross_val_score

# cv=5 → делим данные на 5 частей, проверяем 5 раз
scores = cross_val_score(full, X, y, cv=5)
print("Оценки по фолдам:", scores.round(3))
print("Среднее:", scores.mean().round(3))

```

Пять чисел вместо одного показывают не только средний результат, но и его стабильность. Если оценки скачут (0,60 и 0,80) — модель неустойчива, и одному запуску верить нельзя.

Соревнование класса

Возьмите общий датасет (например, House Prices с Kaggle). Каждый собирает свой Pipeline и настраивает модель, а победитель определяется по средней оценке кросс-валидации — не по одному везучему split. Так соревнование становится честным.

Проверь себя ⓘ

1. Как Pipeline защищает от утечки данных?
2. Зачем масштабировать признаки перед KNN?
3. Чем оценка по кросс-валидации надёжнее одного train/test split?

Глава 11. Метрики и матрица ошибок

Мы всё время мерили качество через ассигасу — долю правильных ответов. Пора узнать, почему одной ассигасы категорически недостаточно и как понять, какие именно ошибки делает модель.

Провокация: 99% и полностью бесполезно

Аналогия: детектор редкой болезни 💡

Болезнь встречается у 1 человека из 100. Сделаем «модель», которая всем говорит «здоров». Её ассигаса — 99%! Но она не нашла ни одного больного и абсолютно бесполезна. Вот почему на несбалансированных данных ассигаса врёт: большой класс перекрывает всё.

Матрица ошибок (confusion matrix)

Чтобы увидеть, где именно модель ошибается, строят матрицу ошибок — таблицу 2×2, которая раскладывает все ответы модели на четыре типа.

	Модель: «да»	Модель: «нет»
На самом деле «да»	Верно (TP)	Пропуск (FN)
На самом деле «нет»	Ложная тревога (FP)	Верно (TN)

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

pred = full.predict(X_test) # full — наш Pipeline из главы 10
cm = confusion_matrix(y_test, pred)
ConfusionMatrixDisplay(cm).plot()
plt.show()
```

Precision, Recall и F1

Из матрицы выводятся три метрики, которые говорят гораздо больше, чем ассигаса.

Метрика	Отвечает на вопрос	Когда важна
Precision	Из тех, кого модель назвала «да», сколько реально «да»?	Когда дорога ложная тревога
Recall	Из всех реальных «да» сколько модель нашла?	Когда дорого пропустить
F1	Баланс между precision и recall	Когда важно и то, и другое

Аналогия: спам-фильтр против детектора болезни 💡

В спам-фильтре важнее precision: лучше пропустить пару спам-писем, чем отправить важное письмо в спам (ложная тревога дорога). В детекторе болезни важнее recall: лучше лишний раз перепроверить здорового, чем пропустить больного. Метрику выбирают под цену ошибки в конкретной задаче.

```
from sklearn.metrics import classification_report

# Сразу все метрики по каждому классу
print(classification_report(y_test, pred))
```

Как выбрать главную метрику

Универсального ответа нет — всё зависит от задачи и цены ошибки. Прежде чем оценивать модель, задай вопрос: что дороже — ложная тревога или пропуск? Ответ и подскажет метрику. Это, пожалуй, главный навык взрослого специалиста по ML.

Проверь себя

1. Почему accuracy 99% может означать бесполезную модель?
2. Чем precision отличается от recall?
3. Для детектора мошеннических операций какая метрика важнее и почему?

Глава 12. Мини-проект №2: «Предсказатель»

Вторая фаза завершается настоящим проектом. Здесь ты один раз проходишь весь путь, который специалисты по данным проходят каждый день: от сырых данных до обоснованного вывода — с обязательным Pipeline.

Полный цикл проекта

Твой проект должен пройти пять шагов. Это скелет любого ML-проекта, и мы будем повторять его до автоматизма.

1. Постановка задачи. Что предсказываем и зачем? Классификация или регрессия?
2. Данные. Выбрать датасет, изучить, почистить (пропуски, выбросы).
3. Baseline. Построить тупую модель как точку отсчёта.
4. Модель в Pipeline. Собрать конвейер и оценить по кросс-валидации.
5. Вывод. Сравнить с baseline, выбрать метрику, объяснить результат словами.

Шаблон проекта

Вот каркас, который остаётся заполнить своими данными. Он объединяет всё из фазы 2.

```
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.dummy import DummyClassifier

# 1. Данные — подставь свой датасет
df = pd.read_csv("МОЙ_ДАТАСЕТ.csv")
X = df.drop(columns=["ЦЕЛЬ"])
y = df["ЦЕЛЬ"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 2. Baseline
base = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
print("Baseline:", base.score(X_test, y_test))

# 3. Pipeline (укажи свои числовые и текстовые столбцы)
num = ["ЧИСЛО1", "ЧИСЛО2"]
cat = ["ТЕКСТ1"]
prep = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                      ("sc", StandardScaler())]), num),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat),
])
model = Pipeline([("prep", prep),
                  ("clf", RandomForestClassifier(random_state=42))])

# 4. Честная оценка
```

```
scores = cross_val_score(model, X, y, cv=5)
print("Модель (кросс-валидация):", scores.mean().round(3))
```

Три слайда для защиты

Оформи результат как мини-презентацию: задача → данные и результат → вывод.
Обязательно сравни свою модель с baseline — это показывает, что ты понимаешь, чего стоит результат.

Как оценивается

Критерий	Баллы	Что оценивается
Постановка задачи	0–2	Чётко, что предсказываем
Данные и Pipeline	0–3	Чистка, корректный split, Pipeline
ML и сравнение с baseline	0–3	Модель обучена и сравнена с базовой
Визуализация и метрики	0–2	Графики и правильная метрика
Объяснение результата	0–3	Вывод словами, понимание ошибок

Главная мысль фазы 2 🎯

Ты умеешь строить модели KNN, деревья, регрессию и случайный лес, собирать их в Pipeline, честно проверять через train/test и кросс-валидацию, сравнивать с baseline и выбирать правильную метрику. Это полный набор навыков классического ML. Дальше — нейросети.